MEMORIA PRÁCTICA 1

Matemática discreta

Grupo 15B

Jimena Zubizarreta Gómez
Pilar Pérez Hernández
2ºB iMAT

INTRODUCCIÓN 2
DECISIONES Y ESTRATEGIAS DE DISEÑO TOMADAS 2
DISEÑO GENERAL DE “IMAT-LAB” 3
Esquema de diseño de la aplicación 3
Errores individualizados 4
Funcionalidades adicionales 4
LIBRERÍA “MODULAR.PY” 5
Ejercicio 1 – es_primo(n:int) → bool 5
Ejercicio 2 – lista_primos(a:int, b:int) → List[int] 5
Ejercicio 3 – factorizar(n:int) →Dict[int,int] 5
Ejercicio 4 – mcd(a:int, b:int)→int + bezout(a:int, b:int)→Tuple[int, int, int] 6
Ejercicio 5 – mcd_n + bezout_n (Opcional) 6
Ejercicio 6 – coprimos(a:int, b:int) → bool 6
Ejercicio 7 -potencia_mod_p(base:int, exp:int, p:int) → int 7
Ejercicio 8 – inversa_mod_p(n:int, p:int)→int 7
Ejercicio 9 -euler(n:int)→int 8
Ejercicio 10 – legendre(n:int, p:int) →int 8
Ejercicio 11-resolver_sistema_congruencias(alist, blist, plist)→tuple[int, int] 9
BIBLIOGRAFÍA 9

INTRODUCCIÓN

Esta primera práctica de Matemática discreta se centra en el diseño e implementación de una librería en Python, modular.py, destinada a resolver problemas de aritmética modular. El proyecto ha implicado la creación de algoritmos matemáticos eficientes y su integración en una interfaz interactiva llamada IMAT-LAB y la validación de su funcionamiento mediante tests unitarios y benchmarks de rendimiento. El desarrollo de este proyecto nos ha permitido aplicar los conocimientos teóricos del temario de la asignatura, diseñar algoritmos de forma eficiente y familiarizarnos con las herramientas de validación (tests) y análisis de rendimiento (benchmarks).

DECISIONES Y ESTRATEGIAS DE DISEÑO TOMADAS

Esta práctica se divide en 3 ficheros principales para que el sistema funcione como se espera además de hacer uso de librerías ya existentes de Python. Estos tres ficheros son imatlab.py, modular.py y excepciones.py. La organización en tres módulos, a excepción del fichero de excepciones, no fue una decisión del equipo, sino un requisito de la práctica.

El módulo modular.py contiene las funciones matemáticas mientras que el imatlab.py gestiona la interacción con el usuario en modo interactivo o la utilización de ficheros de datos cuando se utiliza el modo por lotes. Por tanto, una de las decisiones tomadas por el equipo fue la creación un tercer fichero: excepciones.py. Al hacer esto pudimos tener un control claro de excepciones ya que no se limita únicamente a las exigidas por el enunciado. De este modo, todas las excepciones quedaron centralizadas en el mismo sitio y tenían el mismo formato, proporcionando unos mensajes de error personalizados para cada posible excepción lo que permite generar mensajes y facilita la comprensión de la librería por parte del usuario.

Otra de las decisiones iniciales fue utilizar expresiones regulares como herramienta para validar y analizar los comandos introducidos por el usuario y los ficheros. Al hacerlo así aseguramos que los argumentos y la sintaxis de cada comando se revisase evitando entradas mal formadas, simplificando el proceso e incrementando la eficiencia del programa. Encontramos un caso más complejo que el resto para en una función específica: resolverSistema. Dado que esta función requería una forma distinta de introducir los datos, fue necesario crear una expresión particular e individual para esta función. Esta decisión permitió una validación rigurosa.

Otra de las decisiones que consideramos esenciales fue organizar el código en funciones auxiliares independientes, una por cada comando. De esta manera, la lógica de cada operación queda encapsulada de forma personalizada. Todas cuentan con una validación de argumentos, manejo de errores y llamada a las funciones de modular.py para ejecutarlas. Esta estrategia proporciona un código limpio y hace que el mantenimiento sea más sencillo al no concentrarse toda la lógica en una única función extensa. Al ser funciones pequeñas, resulta más sencillo corregir e identificar posibles errores.

Por último, fue necesario crear un diccionario de correspondencias que relacionara el nombre con el que se invoca a cada función y su implementación correspondiente en el modular.py. Este diccionario facilita la lectura y organización del código, además podría ser extendido en un futuro si se le añaden otras funciones al programa ya que bastaría con incorporar un nuevo par de clave-valor en el diccionario y definir la función auxiliar correspondiente.

Por lo tanto, gracias a la aplicación de diferentes estrategias de diseño, se ha conseguido un alto rendimiento, una mayor eficacia y una experiencia de usuario más satisfactoria, dando lugar a un sistema más claro, estructurado y fácil de mantener.

DISEÑO GENERAL DE "IMAT-LAB"

El diseño general de imat-lab se llevó a cabo de forma exhaustiva y cuidados priorizando la eficacia y la calidad del código. Mas tarde estos dos aspectos fueron probados con pytests.

ESQUEMA DE DISEÑO DE LA APLICACIÓN

En primer lugar, todo el diseño de la aplicación se centra en que funcione tanto en el modo interactivo del usuario como en el modo por lotes. Esta idea fue crucial para que el código quedase limpio y no hubiese repeticiones. Sin embargo, cuando se ejecuta la aplicación, se distingue entre estos dos según los argumentos que mete el usuario, si la longitud de los argumentos es 1, se empleará el modo interactivo. El usuario podrá forzar la salida del modo interactivo invocando Ctrl+C, Ctrl+D o un Enter en blanco. De esta forma el usuario no se queda en un bucle infinito dentro de la aplicación. Por otro lado, el modo por lotes se ejecuta correctamente solo si se proporcionan los ficheros de entrada en modo lectura y de salida en modo escritura para mostrar los resultados de los comandos. Si no se proporcionan esto ficheros saltará un mensaje de error indicando que no se han proporcionado los argumentos válidos.

Sin embargo, independientemente del modo en que se quiera ejecutar el programa, la función run_commands(fin, fout) recibe los manejadores de entrada y salida. En este proceso lee la entrada línea a línea, elimina espacios y saltos de línea, y se detiene cuando encuentra una línea vacía. Cada línea no vacía pasa por la función detectar_funcion(linea), que comprueba que se respete la forma de los argumentos y devuelve una tupla con el nombre del comando y la cadena de argumentos en bruto. A continuación, la función formatear_argumentos se encarga de transformar esa cadena en estructuras de datos que puedan utilizarse en modular.py: listas o enteros, y tres listas en el caso de resolverSistema. Con estos dos elementos, run_commands puede invocar a la función hallar_resultado, que consultará el diccionario de correspondencias y llamará a la función auxiliar que implementa el comando pedido por el usuario o leído del fichero.

Una vez se obtiene el resultado de la función de modular.py a través de las auxiliares, se formatea la salida para que sea fácil de entender por el usuario. Por ejemplo, las funciones booleanas se imprimen como "Sí" o "No", y los diccionarios de los factores de un número se muestran ordenados de menor a mayor. Cualquier condición no contemplada no detendrá la ejecución: se captura y se informa mediante las excepciones NE y NOP.

En la detección del comando solicitado se utiliza detectar_funcion, que aplica una expresión regular a la línea completa. Si la línea está vacía o no encaja con el patrón, se lanzará un NOP con un mensaje explícito, pero el programa seguirá funcionando. En caso contrario, devolverá el identificador del comando y la subcadena exacta con los datos.

Por otro lado, formatear_argumentos tiene dos caminos. En primer lugar, intenta el caso particular de resolverSistema: si la cadena de argumentos coincide con el patrón especial que reconoce tres listas separadas por comas, divide cada bloque, convierte los elementos a enteros y los mete en listas de la misma longitud. Si no se reconoce ese patrón, aplica el camino general para el resto de las funciones: limpia corchetes y espacios, formateando los números a int y devolviendo lo necesario para ejecutar la función.

Finalmente, hallar_resultado, a partir del comando enviado por el usuario, utiliza el diccionario correspondencias para comprobar que esa operación existe y tiene una función asociada, a la cual invoca con los argumentos ya formateados. Este diseño permite añadir nuevas funciones de forma sistemática: se registra el comando en el diccionario, se implementa una función auxiliar y la correspondiente en modular.py, y el resto del flujo no tiene por qué alterarse.

ERRORES INDIVIDUALIZADOS

Imatlab es capaz de distinguir entre errores de formato y uso o errores de naturaleza matemática. Estas excepciones se pueden observar tanto en el código como en el fichero excepciones.py y en los mensajes que se imprimen al usuario. De este modo, estos errores tienen un objetivo doble: que el usuario sepa si debe corregir como ha escrito los comandos o los datos y que la ejecución no se pare cuando se ejecute el modo por lotes.

En la identificación de errores, el proceso siempre sigue los mismos pasos:

1. Detectar_funcion verifica si la expresión cumple el patrón regex, si la línea no se ajusta, lanza una excepción NOP que se captura en run_commands y se muestra al usuario como un mensaje claro: “NOP: El formato de la llamada no es válido”.
2. Formatear_argumentos comprueba si los argumentos tienen el aspecto esperado, si no lo tienen, se lanzará la excepción NOP incluyendo más información sobre porque ha ocurrido este error.
3. Los casos de excepciones matemáticas de capturan en la parte del modular.py.
4. Si el usuario mete un dato inválido, esto también será capturado por el programa, mostrando un mensaje de error: “NOP: Debes meter un número entero”.

El módulo excepciones.py centraliza en dos tipos de excepciones: de formato (NOP) y uso incorrecto además de una tercera excepción para casos individuales que se generan en el apartado del cálculo de funciones. Para mostrar estos errores se ha creado una función de formato para que garantice el formato uniforme de salida. En este caso indicará la función que ha producido el error, la causa de este, el número de argumentos esperados y los realmente recibidos.

En funciones como la inversa o aquellas que utilizan el operador del módulo, existen mensajes personalizados sobre determinados valores que no se pueden utilizar, para ciertos valores que no se pueden utilizar, como es el caso del 0. Otro ejemplo es en la función resolverSistema donde se comprueba que las tres listas tengan loa misma longitud.

La presentación final de los errores está integrada en run_commands donde se capturan tanto las excepciones NOP como NE. De este modo, el programa nunca se detiene por un error, sino que continúa su ejecución mostrando únicamente el mensaje correspondiente.

FUNCIONALIDADES ADICIONALES

Algunas de las funcionalidades adicionales que hemos decidió implementar en nuestro programa incluye las expresiones de patrón general y particular para identificar funciones o las funciones auxiliares que garantizan el formato de los argumentos para que se ejecuten las funciones correctamente.

En lugar de revisar las expresiones introducidas por el usuario o el fichero manualmente, se ha utilizado la librería regex, que permite identificar de forma optimizada patrones de texto complejos que vienen dados de cierta forma. Esta elección de formateo reduce la ambigüedad y evita errores de separación facilitando el rechazo de entradas inválidas.

Sin embargo, esta identificación de patrones no podía ser completamente funcional para cada función y hemos tenido que definir un patrón especial para la función resolverSistema. Esto se debe a que es la única función que recibe 3 listas como datos y no una sola lista o números enteros. A partir de este reconocimiento especial, se verifica que las tres listas tuviesen la misma longitud. Esto garantiza la coherencia de los datos y evita que se formen errores relacionados con el formato o el espaciado.

En tercer lugar, cada función auxiliar valida el número y tipo de argumentos antes de la ejecución, asegurando que se cumpla el contrato definido en cada función del modular.

LIBRERÍA "MODULAR.PY"

EJERCICIO 1 – ES_PRIMO(N:INT) → BOOL

La función es_primo determina si un número entero n es primo. El algoritmo sigue estos pasos:

1. Descarta los casos base: n<=1 no es primo, 2 sí lo es, y cualquier número par mayor que 2 no lo es.
2. Comprueba divisores impares desde 3 hasta raizde(n). Si alguno divide a n, este es compuesto, si no, se concluye que será primo.

Nos hemos basado en la definición de un número primo y en el teorema de la cota de divisores primos, que asegura que todo número compuesto tiene un divisor primo menor o igual que su raíz cuadrada.

Para optimizar el rendimiento, evitamos recorrer todos los enteros hasta n, limitándonos a raizde(n) y descartando números pares.

EJERCICIO 2 – LISTA_PRIMOS(A:INT, B:INT) → LIST[INT]

La función lista_primos recibe dos enteros a y b y devuelve una lista con todos los números primos en el intervalo [a,b). El procedimiento recorre cada entero del rango y, utilizando la función es_primo, comprueba si es primo. Si es el caso, lo añade a la lista de resultados.

Nos hemos basado en la definición de número primo y en Teorema Fundamental de la Aritmética, que garantiza que todo número mayor que 1 puede clasificarse como primo o compuesto.

Para optimizar el rendimiento se aplica la misma estrategia que en el ejercicio anterior ya que se está convocando a esa función.

EJERCICIO 3 – FACTORIZAR(N:INT) →DICT[INT,INT]

La función factorizar recibe un número entero n y devuelve un diccionario cuya clave son los primos que dividen a n y cuyo valor es el exponente con el que cada primo aparece en la descomposición de n.

En este algoritmo implementamos de forma directa el Teorema fundamental de la aritmética, que establece que todo entero mayor que 1 se puede expresar de forma única como producto de potencias de primos. Para calcular la factorización seguimos los siguientes pasos:

1. Contemplamos los casos especiales n € {-1,0,1}, que devuelven un diccionario vacío.
2. Se extraen los factores de 2 mediante un bucle.

3. Se recorren únicamente divisores impares hasta raizde(n), contando cuántas veces dividen a n.
4. Si al final se queda un número mayor que 1, se añade como primo restante.

En cuanto a la eficiencia, en este algoritmo evitamos de nuevo las repeticiones innecesarias limitando la búsqueda hasta la raíz cuadrada y descartando pares después del 2.

EJERCICIO 4 – MCD(A:INT, B:INT)→INT + BEZOUT(A:INT, B:INT)→TUPLE[INT, INT, INT]

La función mcd calcula el máximo común divisor de dos enteros mediante el Algoritmo de Euclides. El procedimiento consiste en ir reemplazando (a, b) por (b, a mod b) hasta que el segundo valor sea cero. En ese momento, el primer elemento es el mcd. Este método está justificado por las propiedades de divisibilidad vistas en clase y asegura que el resultado es el mayor divisor común de ambos números.

La función bezout, implementa el algoritmo de Euclides extendido para obtener, además del mcd, los enteros x e y que verifican la identidad de Bézout:

$$d = ax + by$$

donde d = mcd(a, b). Para ello, utilizamos recursión y la reconstrucción de los coeficientes en cada paso del algoritmo.

Ambas funciones están fundamentadas en el Teorema de Bézout, que garantiza la existencia de esa combinación lineal, y las va,mos a utilizar para cálculos posteriores en la práctica.

EJERCICIO 5 – MCD_N + BEZOUT_N (OPCIONAL)

La función mcd_n generaliza el cálculo del máximo común divisor a una lista de enteros. Está basada en la propiedad asociativa del máximo común divisor que nos permite calcularlo iterativamente con el algoritmo de Euclides de dos en dos (usando las funciones anteriores). Si la lista está vacía devolvemos 0 como caso especial.

La función bezout_n generaliza el algoritmo de Euclides extendido a más de dos números. Devuelve el mcd común a todos y una lista de coeficientes [x1,..,xn] que verifican la identidad de Bezout generalizada:

$$d = a1x1 + a2x2 + \cdots + anxn$$

Para ello, se van incorporando los elementos progresivamente, haciendo el ajuste de coeficientes obtenidos en cada paso.

Ambas de estas funciones se basan en el Teorema de Bézout y en la propiedad asociativa del mcd.

EJERCICIO 6 – COPRIMOS(A:INT, B:INT) → BOOL

La función coprimos determina si dos enteros a y b son coprimos, es decir, si su máximo común divisor es igual a 1. La implementación se basa directamente en la función mcd, ya que la definición establece que dos números son primos relativos (coprimos) si: $mcd(a, b) = 1$

El algoritmo calcula el mcd y devuelve True cuando el resultado es 1 y False si no.

Matemáticamente utilizamos la definición de coprimalidad y el Teorema de Bézout, que garantiza que, si dos enteros son coprimos, existen coeficientes enteros x e y tales que:

$$ax + by = 1$$

EJERCICIO 7 -POTENCIA_MOD_P(BASE:INT, EXP:INT, P:INT) 🕓 INT

La función potencia_mod_p calcula el valor de una potencia módulo p, es decir:

$$base^{exp} (mod\ p)$$

Aplicamos la propiedad:

$$(a \cdot b) mod\ p = \big((a\ mod\ p)(b\ mod\ p)\big)\ mod\ p$$

Porque permite ir reduciendo los cálculos en cada paso y evitar trabajar con números grandes.

Para exponentes negativos usamos que:

$$a^{-k} \equiv (a^{-1})^k\ (mod\ p)$$

Es decir, se calcula la inversa modular de la base y se convierte el problema en uno con exponente positivo. Utilizamos el método de exponenciación rápida porque esto hace que sea más eficiente.

El algoritmo funciona de la siguiente manera:

1. Reduce la base módulo p
2. Si el exponente es negativo, calcular la inversa de la base y transformar el exponente a positivo
3. Inicializa el resultado en 1
4. Mientras el exponente > 0:
    a. Si es impar, multiplicar el resultado por la base y reducir el módulo p
    b. Elevar la base al cuadrado y reducir módulo p
    c. Dividir el exponente entre 2
5. Devolver el resultado

EJERCICIO 8 – INVERSA_MOD_P(N:INT, P:INT)🕓INT

La función inversa_mod_calcula la inversa de un número n módulo p, es decir, un entero x tal que:

$$n \cdot x \equiv 1\ \ (mod\ p)$$

Un número n es invertible módulo p solo si son coprimos, es decir, el máximo común divisor es 1

Se aplica el teorema de Bézout: si el máximo común divisor es 1, existen enteros x, y con:

$$n \cdot x + \ p \cdot y = 1$$

Tomando la congruencia módulo p, queda $n \cdot x \equiv 1\ (mod\ p)$. Por eso el coeficiente x que devuelve Bézout es precisamente la inversa de n.

El algoritmo funciona de la siguiente forma:

1. Si p=0, no existe aritmética modular →error
2. Se llama a la función bezout(n,p) para obtener (d,x,y)

3. Si d != 1, significa que n y p no son coprimos →no hay inversa
4. Si d = 1, el valor de x es la inversa: se devuelve normalizado en [0,p-1] con x%p

EJERCICIO 9 -EULER(N:INT)→INT

La función Euler calcula la función fi de Euler, que cuenta cuántos enteros positivos menores que n son coprimoscon n.

Por definición:

$$\varphi(n) = |\{a \leq k < n : mcd(k,n) = 1\}|$$

Si n es primo, el resultado es simplemente n-1, porque todos los números menores que un primo son coprimos con él.

En general, aplicamos la fórmula multiplicativa de Euler

$$\varphi(n) = n\prod(1 - \frac{1}{p})$$

Donde el producto recorre los primos que dividen a n.

Para usar esta fórmula es necesario factorizar n en primos, lo cual se hace con la función factorizar.

El algoritmo funciona de la siguiente forma:

1. Si n <= 0, devuelve 0
2. Si n es primo, devuelve directamente n-1
3. Si no, se factoriza n
4. Se aplica la fórmula multiplicativa recorriendo los primos obtenidos en la factorización
5. Devuelve el valor final

EJERCICIO 10 – LEGENDRE(N:INT, P:INT) →INT

La función legendre calcula el símbolo de Legendre(n/p) para un número entero n y un primo impar p.

El símbolo de Legendre se define como:

$$\left(\frac{n}{p}\right) = \begin{cases} 0 & n \equiv 0 \pmod{p} \\ 1 & n \text{ es un cuadrado módulo } p, \text{ es decir, } n \equiv k^2 \pmod{p} \text{ para algún entero } k \not\equiv 0 \pmod{p} \\ -1 & \text{en caso contrario} \end{cases}$$

Para calcularlo, se usa el criterio de Euler:

$$(\frac{n}{p}) \equiv n^{\frac{p-1}{2}} (mod\ p)$$

Esto significa:

- Si el resultado es 1 → n es un cuadrado módulo p
- Si el resultado es p-1 (equivalente a -1) →n no es cuadrado módulo p
- Si n = 0 (mod p) → el símbolo vale 0

El algoritmo funciona de la siguiente manera:

1. Verifica que p sea un primo impar (caso especial: error si p=2)
2. Reduce n módulo p
3. Si n=0, devuelve o
4. Calcula $r = n^{\frac{p-1}{2}} (mod\ p)$ usando la función potencia_mod_p
5. Devuelve 1 si r=1, -1 si r=p-1

El hecho de usar el criterio de Euler simplifica el problema de comprobar si un número es cuadrado módulo p a un cálculo lo más eficiente posible.

EJERCICIO 11-RESOLVER_SISTEMA_CONGRUENCIAS(ALIST, BLIST, PLIST)→TUPLE[INT, INT]

La función resuelve un sistema de congruencias lineales de la forma:

$$aix \equiv bi\ (mod\ pi), i = 1, \dots., n$$

Donde los módulos pi son coprimos dos a dos. El resultado es un par (r,m) tal que la solución general del sistema es:

$$x \equiv r\ (mod\ m), \quad m = p1 \cdot p2 \cdots pn$$

Para pasar de $aix \equiv bi\ (mod\ pi)$ a la forma estándar, se multiplica por la inversa modular de ai (si existe), de modo que:

$$x \equiv bi \cdot ai^{-1}\ \ (mod\ pi)$$

Una vez reducidas todas a la forma $x \equiv xi\ (mop\ pi)$, se aplica el Teorema Chino del Resto.

El algoritmo funciona de la siguiente manera:

1. Verifica que las tres listas tengan la misma longitud
2. Calcula el producto total de los módulos m
3. Para cada congruencia (a,b,p):
    a. Calcula la inversa modular de a (mod p)
    b. Obtén $xi = b \cdot a^{-1} (mod\ p)$
    c. Calcula Ni=m/p
    d. Halla la inversa modular de Ni (mod p)
    e. Acumula en la solución $xo = xo + xi \cdot Ni \cdot Ni^{-1} (mod\ m)$
4. Devuelve (x0, m)

BIBLIOGRAFÍA

regex101. (s. f.). *Regular Expressions 101*. Recuperado de https://regex101.com/

Departamento de Matemática Aplicada. (s. f.). *Teoría de Números*. Grado en Ingeniería Matemática e Inteligencia Artificial (IMAT), Escuela Técnica Superior de Ingeniería (ICAI), Universidad Pontificia Comillas.